

The Ingestion Verdict

A concrete call on Apify, and what it means for Registry Sync, Source Intelligence Analyzer, the Job Ingestion pilot, the CSV registry work, and the build sequence from here.

Base evidence **The Ingestion Gap, 2026-09-26**

New research **Apify marketplace, pricing — external, sourced**

Scope **decision + plan only — nothing modified or activated**

Contents

01 Legend

02 Apify decision

03 Real benchmark plan

04 Existing automations

05 CSV company files

06 More ATS adapters?

07 Production architecture

08 Registry field re-evaluation

09 P0/P1/P2 re-prioritization

10 LinkedIn

11 Final decision

· How to read this report

Every claim below is tagged so you can tell what's solid ground and what's a call being made on your behalf.

FACT verified in our code/DB/live n8n instance, or sourced external research

INFERENCE a reasoned conclusion built on facts, not independently verified

RECOMMENDATION what I'd do — actionable, but yours to override

FOUNDER DECISION requires your explicit sign-off before anything proceeds

01 Apify decision

The question conflates four different activities. They get four different answers.

USE CASE	CURRENT STATE	VERDICT
Apify for company/source discovery	FACT Already adopted — Registry Sync's Apify mode reads a pre-completed dataset to add candidate rows to <code>company_sources</code> .	Keep as-is. Low risk, already tested, not the bottleneck.
Apify for career-page / job extraction (non-LinkedIn)	FACT Not used anywhere in our code today. Third-party marketplace actors exist that detect the ATS behind a careers page (Greenhouse, Lever, Ashby, Workday, SmartRecruiters, Rippling, Breezy HR, Personio, BambooHR) and extract structured jobs, priced roughly \$0.55–\$1 per 1,000 job records.	YES, but only as a benchmarked, bounded tier. Not yet a major tier — no evidence yet that it works well against <i>our specific</i> registry.
Apify Actors as a production ingestion tier generally	INFERENCE Generalization of the row above — running a marketplace actor on a schedule, writing its output into <code>jobs</code> .	Same verdict, same condition: passes through the identical Normalize→Validate→Dedup pipeline every other tier passes through. Never a trusted direct writer.
LinkedIn -via-Apify	FACT Prohibited by AGENTS.md §7 as written today; two shadow n8n workflows already exist live, untracked, exploring exactly this.	Decided separately — Section 9. Not part of this Apify verdict.

The concrete outcome you asked for

RECOMMENDATION

YES, but only as a fallback / limited tier — for now. Not "no," and not "yes, major tier" either, because neither is earned yet. Here's the exact reasoning:

- **FACT** The nominal cost is low enough that trying it is nearly risk-free: \$0.55–\$1 per 1,000 job records for career-page extraction, sourced from actual current Apify Store listings.
- **INFERENCE** Marketplace actor quality/reliability against Gulf/Lebanon custom career sites specifically is unverified — the actors' own marketing claims to support "any" career page, but the audit already found 402 of 593 sources have an unclassified or free-text `ats_provider`, meaning many are exactly the messy, non-standard case marketplace actors are least tested against.
- **FACT** Search results show the same nominal LinkedIn-scraping task priced anywhere from \$1.50 to \$12 depending on which listing you pick — this is a marketplace of independent developers, not one vendor with one SLA. That volatility applies to career-page actors too, not just LinkedIn ones.
- **RECOMMENDATION** Committing to "major tier" without evidence would repeat the exact mistake "The Ingestion Gap" warned against for AI agents: reaching for a new dependency before measuring it. The fix is the same one that report already prescribed — a bounded pilot, not more analysis. That pilot is Section 2.

The upgrade path is explicit, not vague: if the benchmark in Section 2 clears the thresholds defined there, Apify graduates from limited/fallback tier to a major tier for the 87% bucket. If it doesn't, we've spent under \$50 and a few hours finding that out — not months building the wrong thing.

Sources: apify.com/pulsedata/career-page-jobs-scraper · apify.com/dami_studio/career-site-jobs-scraper · apify.com/maged120/company-careers-jobs-scraper · apify.com/pricing.

02 Real benchmark plan

Bounded, read-only, against our actual registry. No production writes.

Sample: 80 companies from the difficult 87%

STRATUM	TARGET N	WHY INCLUDED
suitable_public_html_subject_to_review	40	The largest bucket (358 live) — the core question this benchmark exists to answer
automation_eligibility = unknown	20	The active Source Intelligence backlog (95 live); includes sources it has probed without confidence
needs_manual_review, suitable_public_html subset	10	Tests whether "needs research" and "needs a different mechanism" are actually the same 185 rows or two different problems
manual_only / no_official_source_found	5	Small control group — confirms these 72 are genuinely unreachable rather than under-researched
Already-observed by Source Intelligence, medium/low confidence	5	Direct head-to-head: does Apify solve exactly what our own classifier couldn't?

Distribute proportionally across the five live markets (roughly UAE 26%, Lebanon 22%, Kuwait 20%, Saudi 20%, Qatar 13%, matching the live `target_country` split) so the result isn't accidentally skewed toward one region's web conventions.

Per-company test protocol

1. **Pull existing signal** — read-only `SELECT` against `source_intelligence` for this source's most recent observation (`ats_provider`, `automation_eligibility`, `confidence`, `ingestion_type`). No write.
2. **Direct first-party check** — visit the recorded `official_careers_url`: does it 200, is it a JS-only shell, is a job list visible to a human eye, is there an ATS fingerprint the classifier missed.
3. **Apify actor run, sandboxed** — call the Apify API directly against a small, pre-approved test budget (outside n8n, outside any production credential, results written to a local scratch file/dataset only — never to `company_sources` or `jobs`). Run one career-page actor and, where the site looks custom, one generic AI/content-extraction actor.
4. **Record per company**: jobs found (count), 3–5 jobs manually spot-checked for accuracy, whether the apply URL is canonical/first-party vs. a redirect, a freshness signal (posted/updated date present and recent), duplicate risk (same job surfaced by two methods), extraction quality (fields populated correctly vs. garbage), failure mode (blocked/403, JS-required-empty, unsupported ATS, timeout, genuinely zero open roles, other), and the actor's per-result cost for this run.

FOUNDER DECISION

This benchmark spends real (small) money — call it under \$50 given the per-1,000-result pricing found. It needs an approved test budget and a throwaway Apify account/API token before step 3 can run. Nothing else in this report requires spend approval; this is the one exception.

Metrics computed at the end

METRIC	DEFINITION
Coverage %	Sampled companies where ≥ 1 real job was found, $\div$ sample size
Valid job %	Manually-confirmed-accurate jobs $\div$ total jobs extracted
Freshness %	Jobs with a discoverable, recent posted/updated date $\div$ jobs found
Failure rate	Sampled companies where no method produced anything usable, $\div$ sample size
Cost per 100 companies	Extrapolated from actor pricing $\times$ average jobs-per-company observed in the sample
Cost per 1,000 companies	Same, $\times 10$ — sanity-checked, not just linearly assumed
% needing browser fallback	Sampled companies where only a JS-rendering/full-browser approach worked at all
% remaining manual/unautomatable	Sampled companies where nothing — including Apify — found anything usable

03 Existing automations — what happens to them

A. Registry Sync

KEEP BUT MODIFY

It's already the correct pattern — converge every discovery input (manual, Sheets, Apify dataset-read) into one RPC write path. That doesn't change. What modifies:

- **Keep importing CSV seed lists?** Yes — see Section 4. CSVs remain valid seed input; Registry Sync is the write path regardless of which discovery method feeds it.
- **Use Apify for company discovery?** Yes — already does, via dataset-read. No architectural change needed, just keep using it.
- **Use Apify to resolve missing career URLs/ATS?** **INFERENCE** No — that's classification, not registration. Keep the responsibility split: Registry Sync adds/updates rows, Source Intelligence (or its successor) classifies them. Blurring this would recreate the free-text-drift problem already found in `ats_provider`.
- **Store provider capability in `company_sources`?** Yes — this is the Section 7 schema evolution. Registry Sync becomes one of several writers of those new fields once they exist; that's additive, not a Registry Sync rebuild.

B. Source Intelligence Analyzer

KEEP BUT REDUCE ITS ROLE

Not kept just because it's already built — here's the actual case for keeping a smaller version of it:

- **FACT** It costs nothing to run (a single plain HTTP GET, no per-call fee), unlike any Apify tier.
- **FACT** Per "The Ingestion Gap," it has produced **zero high-confidence classifications across 60 real observations** — its auto-promotion path is tested but has never fired on real data.
- **INFERENCE** Since a good Apify career-page actor already does "detect the ATS behind this page" as a side effect of extraction, most of this workflow's original job — figuring out `ats_provider` — is now redundant with what a paid tier would discover anyway, for any source that reaches that tier.

Re-scope it to: a free, first-pass triage filter that runs before any source is allowed to consume paid Apify budget — catching the easy, obvious cases (a source that's plainly Greenhouse/Lever/Workable but mistagged) at zero cost, and routing everything else straight to the benchmarked tier instead of retrying the same free GET every 3 days indefinitely. That's a real, useful, narrower job — not the mechanism that solves the 87% problem, which was never realistically its job.

Separately, **RECOMMENDATION** : spend a small amount of time understanding *why* it never reaches high confidence — its provider-fingerprint list may simply be miscalibrated for this region's HR platforms. Worth a quick root-cause look, independent of the Apify decision.

C. Job Ingestion — Pilot Orchestrator

KEEP BUT MODIFY — unchanged from "The Ingestion Gap"

Activate now, query-driven against the 17 known-good sources, scheduled, Error Handler attached — this doesn't wait on the Apify decision at all; it's free, already-built value. Later, once the benchmark names a winning actor, this workflow's `Route By ATS Type` switch gets a new branch for sources whose capability record says `requires_provider = apify`. That's a future, evidence-gated modification, not a now one.

D. Error Handler

KEEP AS-IS (structurally) — wire it up

Nothing about the Apify analysis changes this one. It was already P1 in the prior audit (wire the notification boundary to a real channel, attach to every scheduled workflow) and stays exactly there.

Where Apify enters the pipeline — checking your proposed flow

Your proposed flow is directionally correct:

```
Registry → Source Router → ATS direct OR Apify OR browser fallback → Normalize → Validate → Deduplicate → Canonicalize → jobs
```

Two refinements, both real, not cosmetic:

- The router should read a **Source Capability Registry** (the extended `company_sources`, Section 7), not the raw registry — it needs to know *which* Apify actor and whether a browser is required, not just that "Apify" is an option.
- **INFERENCE** Apify doesn't map cleanly to one tier from the original framework — the same marketplace actors can cover what was "Tier 1 extension" (a known ATS we haven't built an adapter for), "Tier 2" (generic HTML), and "Tier 4" (AI-first extraction) all at once. Post-benchmark, expect Apify to absorb multiple tiers rather than sit as a single discrete branch — see the full diagram in Section 6.

04 Do we keep our CSV company files?

Yes — mostly. Nothing here is thrown away without evidence.

QUESTION	ANSWER
Which CSV data still useful?	FACT The International Remote market rows — confirmed in "The Ingestion Gap" as never imported into the live database despite being a named target market. That data has real, unconsumed, actionable value sitting in CSV form right now.
Do the CSVs become seed data?	Yes — that is exactly and only what they are now. The five market CSVs (Lebanon/UAE/Qatar/Saudi/Kuwait) have already been fully absorbed into <code>company_sources</code> ; their remaining value is an audit trail, not a live source of new information.
Does <code>company_sources</code> stay central?	Unambiguously yes. It's the live, RLS-capable, foreign-keyed table everything downstream depends on. A CSV file can never be that — no live app can query it safely or concurrently.
Does any of that work become obsolete?	INFERENCE No wholesale obsolescence. The 588 rows of real company/URL research across five markets remain the substance of the registry today — that's not wasted, it's literally what <code>company_sources</code> is. What <i>is</i> obsolete: the narrative <code>discovery-report.md</code> summarizing that research (already flagged stale in the prior audit), and treating CSV files as something to keep hand-editing going forward — Registry Sync, Source Intelligence, and the future Apify tier are the live successors to that manual CSV-editing workflow.

05 Should we still build more ATS adapters?

SmartRecruiters specifically — delay it. This is a direct reversal of "The Ingestion Gap"'s sequencing, and it's worth saying so plainly.

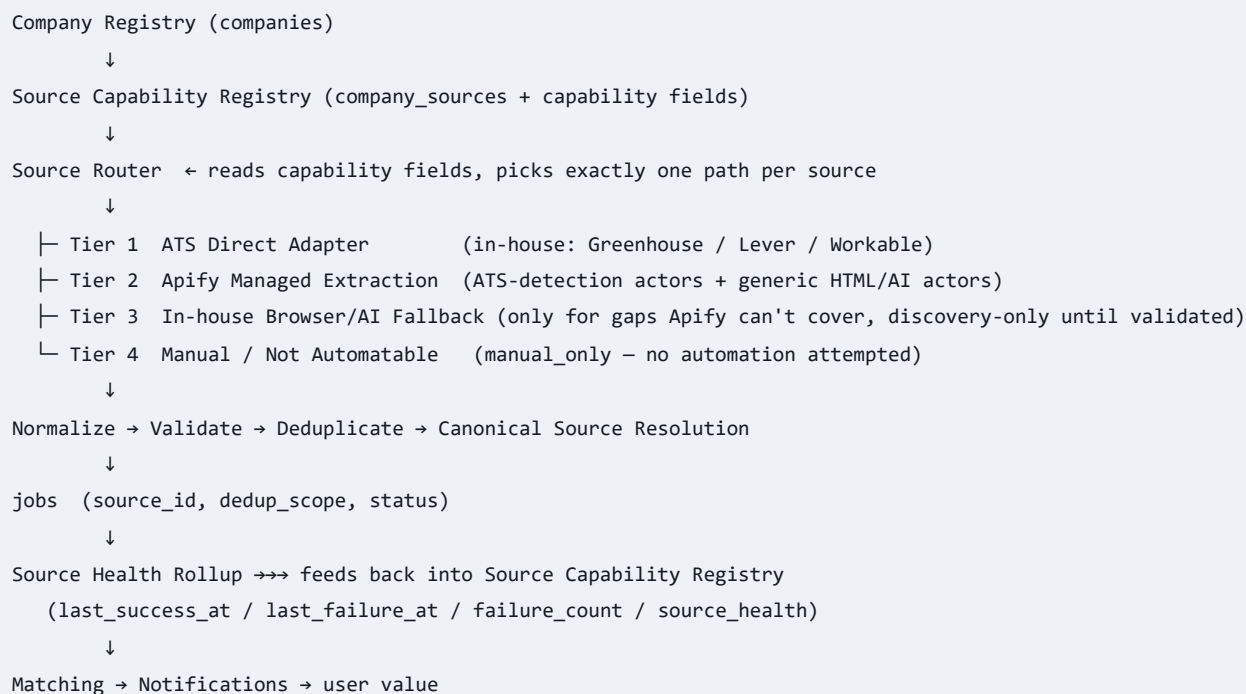
	BUILD IN-HOUSE	USE APIFY (PENDING BENCHMARK)
Coverage today	4 verified SmartRecruiters sources (0.67% of 593)	Same 4, plus Ashby/Workday/Rippling/Breezy HR/Personio/BambooHR support in the same actor family — a wider net for near-zero extra effort
Marginal cost per run	Zero once built, forever	~\$0.55–\$1 / 1,000 job records, ongoing
Engineering time	Real — comparable to the Greenhouse/Lever/Workable adapters, each of which carries its own retry loop and test suite	Near zero — a router branch and a pinned actor id, once validated
Control / reliability	Full control, no vendor dependency	Depends on an independent marketplace developer's actor staying maintained
Vendor lock-in	None	Mitigated by pinning a specific actor per source (<code>provider_actor_id</code> , Section 7) — swappable without touching other sources

RECOMMENDATION

Delay SmartRecruiters (and any other new in-house adapter) until after the benchmark. Spending real engineering time on 0.67% of the registry when a few cents of Apify spend might already cover it — and five other ATS families besides — is exactly the "investing blindly" you said you want to avoid. Only build an adapter in-house afterward, and only for a provider that is (a) high-volume in the registry and (b) shown by the benchmark to be unreliable or expensive via Apify.

Source: apify.com/maged120/company-careers-jobs-scraper — lists Greenhouse, Lever, Ashby, Workday, SmartRecruiters, Rippling, Breezy HR, Personio, BambooHR as supported.

06 Production architecture — if Apify performs well



What happens on each outcome

EVENT	BEHAVIOR
Source succeeds	Write jobs; <code>last_success_at</code> updated; <code>failure_count</code> reset to 0; <code>last_job_seen_at</code> updated; <code>source_health = healthy</code> .
Source fails temporarily	Retry within the run (existing bounded backoff). If still failing: increment <code>failure_count</code> , set <code>last_failure_at</code> , <code>source_health = degraded</code> . Existing jobs are not closed — the existing "never close on a failed/truncated fetch" safeguard holds.
Source repeatedly fails	<code>failure_count</code> past a threshold (e.g. 3–5 consecutive runs) → <code>source_health = unhealthy</code> , auto-paused from the router (stops burning Apify spend on a dead source), flagged for human review. Jobs age out via <code>expires_at</code> rather than being force-closed; only close outright after a longer confirmed-unreachable window (e.g. 14–30 days).
Source is manual-only	Never enters the router at all — excluded by its own capability record.
Source has no usable jobs (fetch succeeded, zero listings)	This is a success , not a failure — a company can legitimately have no open roles right now. Update <code>last_success_at</code> , keep <code>healthy</code> , and this is the one case where closing that source's prior jobs is correct. Flagging this: the pipeline must distinguish "fetch failed" from "fetch succeeded, found nothing" cleanly at every tier — worth a direct verification pass, not an assumption.

07 Source registry field re-evaluation

Re-checked against the Apify decision specifically — not just restated.

FIELD (FROM THE AUDIT)	STILL NEEDED?	WHY, GIVEN APIFY
adapter_key	YES	Router needs to know exactly which in-house adapter <i>or</i> Apify actor to invoke, regardless of tier.
requires_browser	YES	Avoids sending plain-HTTP-capable sources through an expensive browser-based actor — direct cost control.
requires_provider	YES	Sharpens to essentially "apify none" — marks which sources carry ongoing third-party cost.
discovery_only	YES	Unchanged — marks sources that only ever feed discovery, never a direct job write.
canonical_priority	YES — more so now	Once an in-house adapter and an Apify actor could both produce the same job, you need a tie-break rule: in-house ATS adapter > Apify ATS-detected extraction > Apify generic HTML/AI extraction > any discovery/aggregator signal.
last_success_at / last_failure_at / failure_count / last_job_seen_at / source_health	YES — more so now	With Apify, this stops being observability-only and becomes cost control — it's what stops the router from repeatedly paying to hit a dead or blocked source.

Proposed additions

FIELD	VERDICT	WHY
provider_name	Add	Which provider services this source (<code>apify</code> / <code>in_house</code> / <code>none</code>) — required the moment more than one provider exists.
provider_actor_id	Add	Pins the exact benchmark-winning actor + version per source — reproducible cost, and lets one bad actor be swapped without touching other sources. This is the concrete anti-lock-in mechanism.
ingestion_strategy	Skip as a free-standing field	Would duplicate <code>adapter_key</code> + <code>requires_provider</code> + <code>requires_browser</code> . Adding it as its own hand-set string risks repeating the exact free-text drift already found in <code>ats_provider</code> (59 uncontrolled values). If wanted at all, derive it, don't store it.
provider_cost_class	Add	Cheap enum (free/cheap/moderate/expensive) computed from the pinned actor's known pricing — lets a budget guard skip or flag expensive sources instead of silently spending.
last_provider_run_at	Add	Distinct from <code>last_success_at</code> / <code>last_failure_at</code> — tracks when money was last spent on this source regardless of outcome, so expensive sources can be rate-limited independently of free in-house adapters.

FIELD	VERDICT	WHY
canonical_apply_source (<i>not in the original list</i>)	Add	Records whether this source's job URLs are genuinely first-party (company's own apply flow) or third-party/redirect. Matters directly for AGENTS.md's product preference for first-party application paths, and lets matching/UI show "apply on company site" vs. "apply via a discovery source" distinctly.

No migration created here — conceptual only, per your instruction.

08 P0 / P1 / P2 re-prioritization

PRIORITY	ITEMS
P0	• Activate Job Ingestion Pilot Orchestrator (17 sources, query-driven, scheduled) — unaffected by Apify, free value on the table • Reconcile CV Analysis Worker repo/live drift — production is running an unversioned "v2" • Freeze/reconcile the shadow n8n workflows (AI scraper agent, LinkedIn-via-Apify enrichment, Google Maps leads, Lebanon Tech Hiring Intelligence) — more urgent now, not less, since this report is about to bless Apify as a real tier while those workflows already explore it ungoverned • Run the bounded, read-only Apify benchmark (Section 2) — every other Apify-related decision is downstream of it
P1	• Begin the minimal matching worker against real pilot jobs — unblocked by ingestion, was already tied for the highest product risk • Source Intelligence Analyzer re-scope (Section 3) + root-cause why it never reaches high confidence • Error Handler wired to a real channel, attached to every scheduled workflow • Source-capability registry migration (Section 7 fields) — sized to what the benchmark actually finds • Source health tracking (same migration) • International Remote registry gap — cheap, real, currently zero coverage
P2	• SmartRecruiters / any new in-house ATS adapter — explicitly delayed until after the benchmark (Section 5) • Cross-source dedup — only matters once ≥ 2 tiers are live in production • Cover-letter generation and application-sending flows — sequenced after matching exists and jobs flow reliably • Splitting the shared n8n instance — real, but lower urgency than freezing the shadow workflows' content • LinkedIn-via-Apify — parked entirely pending Section 9, not on this ladder at all

09 LinkedIn

Kept fully separate, as instructed. Not decided here.

- **What LinkedIn-via-Apify technically enables:** **FACT** third-party actors query LinkedIn's own public job-search results for postings matching a title/location/company, returning structured fields (title, company, location, description, apply/job URL, posted date) — scraping LinkedIn's own publicly-served listings, not requiring our users to log in at all.
- **How this differs from scraping authenticated data:** **INFERENCE** no login as any real person, no session/cookie belonging to a user or company, no private profile/connection/messaging data touched — only what LinkedIn serves an anonymous visitor searching jobs (subject to LinkedIn's own bot-detection, a reliability risk, separate from the compliance one).
- **What the current project rule prohibits:** **FACT** AGENTS.md §7 states plainly "Never scrape LinkedIn" and "Never automatically apply through LinkedIn," with no carve-out for public-listing discovery. Read literally, it prohibits the activity above too — the public/anonymous distinction is being proposed by this analysis, not something the rule already accepts.
- **What decision would be required before use:** **FOUNDER DECISION** an explicit, deliberate amendment to AGENTS.md §7 naming exactly what's permitted (e.g. public search results only, original apply link preserved, no login, no auto-apply, no profile/connection data) — ideally after a real legal review of LinkedIn's User Agreement and the relevant regulatory/litigation history around public-listing scraping, which varies by jurisdiction and shouldn't be settled by engineering judgment alone. Someone also needs to own the ongoing compliance risk and the Apify spend.

UNTIL THAT DECISION IS MADE

Do not activate, expand, or productionize the two LinkedIn-adjacent shadow workflows already found live. This report does not recommend changing the rule — it only describes what a future decision would need to contain.

Source: apify.com/bebity/linkedin-jobs-scraper · [use-apify.com best-LinkedIn-scrapers](https://use-apify.com/best-linkedin-scrapers) overview.

10 Final decision

This week

- **DO** Activate Job Ingestion Pilot Orchestrator — query-driven, scheduled, against the 17 existing sources.
- **DO** Reconcile CV Analysis Worker repo/live drift.
- **DO** Freeze and reconcile the shadow n8n workflows — get eyes on what they actually do.
- **DO** Run the bounded, read-only Apify benchmark against ~80 sampled sources (needs a small approved test budget — Section 2).

Not this week

- **DON'T** Build SmartRecruiters or any new in-house ATS adapter.
- **DON'T** Commit Apify to any production workflow, or call it a "major tier," yet.
- **DON'T** Touch LinkedIn-via-Apify in any way pending Section 9.
- **DON'T** Start cover-letter or application-sending flows — nothing exists yet to feed them.
- **DON'T** Run the source-capability-registry migration yet — design it once, from benchmark results, not twice.

Direct answers

Which automation do we modify first?	Job Ingestion — Pilot Orchestrator. It's the only one where "modify" means "unlock real, already-tested value this week," with zero new dependencies.
Benchmark Apify now or later?	Now. It's cheap, bounded, and read-only, and every other open question in this report (SmartRecruiters timing, tier design, schema specifics, cost projections) is downstream of it.
SmartRecruiters before or after?	After. Spending real engineering time on 4 sources (0.67% of the registry) before checking whether a fraction of a cent per company already covers it is the blind investment you said you want to avoid.
When does the matching worker start?	Immediately after the Pilot Orchestrator activates this week — P1, not blocked by Apify at all. Get the first true company→jobs→matching→user-value slice running as soon as possible, independent of how the benchmark turns out.

Full sequence, today to a real end-to-end flow

1. Activate Pilot Orchestrator (17 sources, scheduled, Error Handler attached).
2. Reconcile CV Analysis Worker and the shadow workflows — governance, not new capability.
3. Run the Apify benchmark (80 sources, bounded budget, read-only).
4. Build the minimal matching worker against real pilot jobs — in parallel with step 3, not blocked by it.
5. Using benchmark results, design and apply the additive source-capability-registry migration (Section 7), sized to what was actually learned.
6. Wire the winning Apify tier into the Job Ingestion router as a new branch, passing through the same validation every other tier passes.
7. Decide on SmartRecruiters or any other in-house adapter only for whatever the benchmark shows Apify handles poorly.
8. Begin cover-letter and application-sending flows once matching and a wider job set are both live.

9. Resolve the International Remote registry gap and the LinkedIn/Apify founder decision on their own tracks — not blocking anything above.

EXPLICITLY: THE PREVIOUS SEQUENCE CHANGES

"The Ingestion Gap"'s Implementation Sequence, step 3, said to add a SmartRecruiters adapter next. That recommendation predates knowing Apify marketplace actors already cover SmartRecruiters (and five other ATS families) for a fraction of a cent per company. It is superseded: the Apify benchmark now sits before any further adapter work, not after it. The tier boundaries also loosen — what was cleanly "Tier 2 generic HTML" and "Tier 4 AI/browser fallback" in that report may collapse into a single Apify tier once benchmarked, rather than staying as separate build efforts.

Built on the confirmed evidence in "The Ingestion Gap" (2026-09-26) plus new, sourced external research into Apify's marketplace and pricing conducted for this report. No code, database, credentials, schedules, branches, or n8n workflows were modified, activated, created, or deleted in the production of this report.